

Flow control under pressure

We ran four hardened flow-control scenarios against openai/gpt-oss-20b on one H100-backed vLLM replica. The saturation sweep set the load range, then each scenario ran three complete repeats.

What the benchmark showed

The benchmark showed flow control handling four production traffic patterns under load. Each scenario isolated a different behavior across shared endpoint pressure, priority differentiation, same-priority fairness, and batch inversion prevention.

TEST 1

One endpoint, doubled load
The endpoint continued routing requests to the same vLLM runtime as aggregate concurrency increased.

TEST 2

Priority differentiation
Premium traffic stayed steadier while standard traffic absorbed more queue time.

TEST 3

Same-priority fairness
Premium A spiked, while premium B and C continued in the same priority lane.

TEST 4

Priority inversion prevention
Batch traffic surged, but higher-priority traffic kept the better scheduling position.

How flow control was exercised

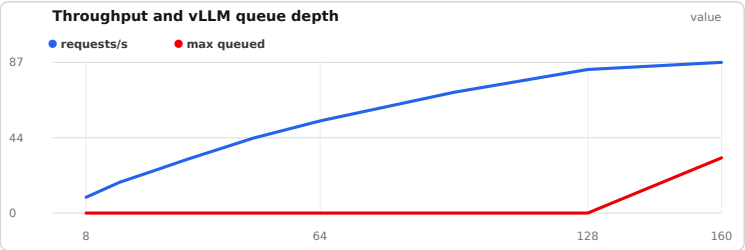
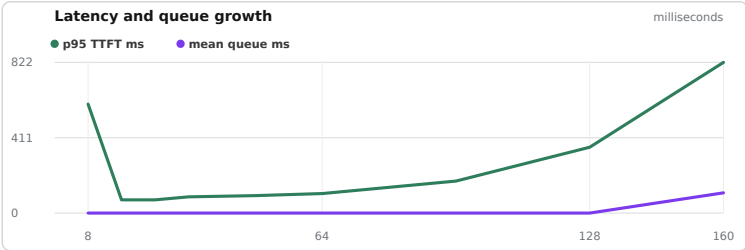
Each request used the same endpoint. The headers told endpoint-picker which priority lane and fairness bucket to use.

TRAFFIC	OBJECTIVE	PRIORITY	MEANING
Premium	gpt-oss-premium	100	Interactive traffic with the strongest predictability target.
Standard	gpt-oss-standard	0	Normal traffic that can absorb more queue time under pressure.
Batch	gpt-oss-batch	-10	Low-priority background work.

- Model: openai/gpt-oss-20b.
- Runtime: one H100-backed vLLM replica.
- Traffic shape: 512 input tokens and 128 output tokens.
- Measurement: client latency, endpoint-picker queue time, and vLLM running and queued request gauges.

Where pressure began

The sweep raised one standard tenant from 8 to 160 concurrent requests. vLLM queue depth appeared at 160 concurrent requests, which gave us the pressure range for the scenario tests.



CONCURRENCY	REQUESTS/S	P95 TTFT	P95 LATENCY	MEAN QUEUE	MAX QUEUED
8	9.1	595 ms	1,345 ms	0.0 ms	0
16	17.8	72 ms	898 ms	0.0 ms	0
24	24.3	72 ms	989 ms	0.0 ms	0
32	31.0	89 ms	1,046 ms	0.0 ms	0
48	43.4	95 ms	1,130 ms	0.0 ms	0
64	53.4	106 ms	1,269 ms	0.0 ms	0
96	70.0	175 ms	1,441 ms	0.0 ms	0
128	83.2	359 ms	1,659 ms	0.0 ms	0
160	87.3	822 ms	2,265 ms	110.2 ms	32

At 128 concurrent requests, vLLM reached its configured running-request limit with 0 queued requests. At 160 concurrent requests, max queued requests reached 32 and mean endpoint-picker queue time reached 110.2 ms.

Same concurrency, different workload cost

We evaluated five input-token targets and output-token caps with a saturation sweep only. The sweep showed that the same concurrency setting can create very different throughput and p95 time to first token.

Throughput at 160 concurrent requests

Short interactive 256 / 64	165.1 rps
Baseline 512 / 128	88.3 rps
Longer prompt 1024 / 128	84.8 rps
Longer output-token cap 512 / 512	27.2 rps
Heavier mixed 2048 / 256	40.0 rps

P95 time to first token at 160 concurrent requests

Short interactive 256 / 64	568.5 ms
Baseline 512 / 128	688.6 ms
Longer prompt 1024 / 128	760.7 ms
Longer output-token cap 512 / 512	5,990.7 ms
Heavier mixed 2048 / 256	2,406.5 ms

PROFILE	INPUT TARGET / OUTPUT CAP	FIRST QUEUED AT CONCURRENCY	REQUESTS/S	P95 TTFT	P95 LATENCY	MEAN QUEUE	MAX QUEUED
Short interactive	256 / 64	160	165.1	569 ms	1,264 ms	46.7 ms	27
Baseline	512 / 128	160	88.3	689 ms	2,093 ms	102.2 ms	28
Longer prompt	1024 / 128	160	84.8	761 ms	2,259 ms	90.7 ms	32

PROFILE	INPUT TARGET / OUTPUT CAP	FIRST QUEUED AT CONCURRENCY	REQUESTS/S	P95 TTFT	P95 LATENCY	MEAN QUEUE	MAX QUEUED
Longer output-token cap	512 / 512	160	27.2	5,991 ms	11,892 ms	55.6 ms	32
Heavier mixed	2048 / 256	160	40.0	2,407 ms	5,746 ms	119.1 ms	32

At the same concurrency, different token profiles can change throughput and latency. The four scenario tests use the baseline profile: 512 input-token target and 128 output-token cap.

Where queue time concentrated

Each scenario pairs the time-series view with the traffic-class data below it.

TEST 1

One endpoint, doubled load

35 max queued

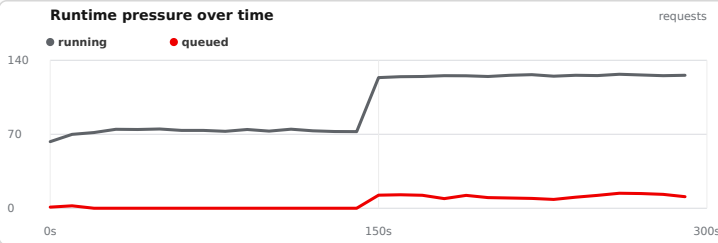
Expected signal observed

Premium B joins halfway through. Aggregate concurrency reaches saturation, so endpoint-picker queue time appears.

Runtime pressure over time

requests

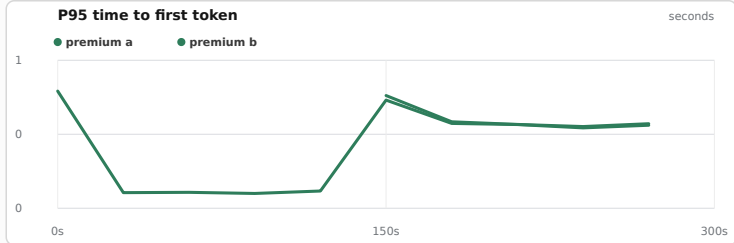
● running ● queued



P95 time to first token

seconds

● premium a ● premium b



● premium a

● premium b

46 ms

114 ms

TRAFFIC	P95 TTFT	P95 LATENCY	MEAN QUEUE	MEAN 200	NON-200
● premium a	546 ms	1,957 ms	46 ms	15,794	0
● premium b	600 ms	2,011 ms	114 ms	6,590	0

TEST 2

Priority differentiation

31 max queued

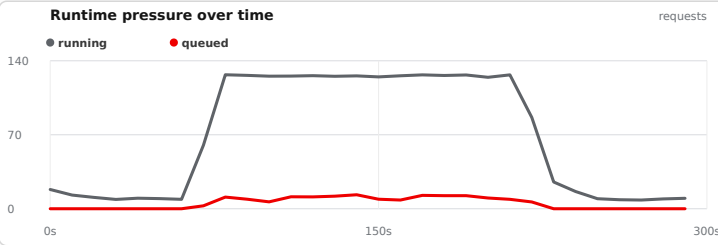
Expected signal observed

Standard traffic ramps up. Premium maintains lower queue time while standard carries more pressure.

Runtime pressure over time

requests

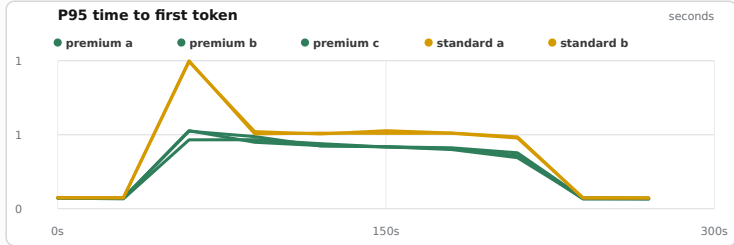
● running ● queued



P95 time to first token

seconds

● premium a ● premium b ● premium c ● standard a ● standard b



● premium a

● premium b

● premium c

● standard a

● standard b

36 ms

37 ms

36 ms

91 ms

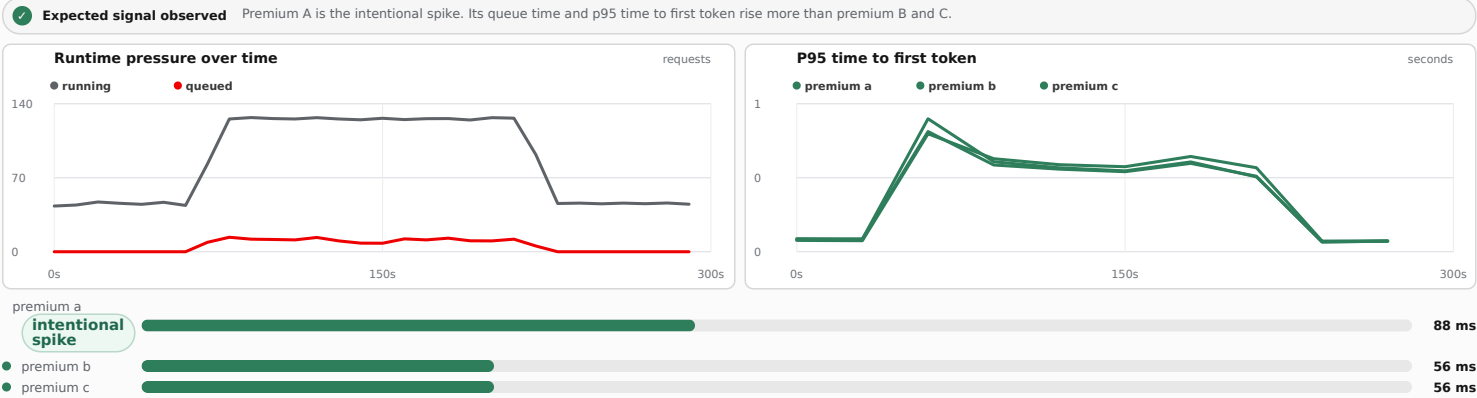
92 ms

TRAFFIC	P95 TTFT	P95 LATENCY	MEAN QUEUE	MEAN 200	NON-200
● premium a	417 ms	1,829 ms	36 ms	1,433	0
● premium b	416 ms	1,826 ms	37 ms	1,186	0
● premium c	420 ms	1,830 ms	36 ms	1,171	0
● standard a	568 ms	1,976 ms	91 ms	7,124	0
● standard b	575 ms	1,983 ms	92 ms	7,066	0

TEST 3

Same-priority fairness

32 max queued

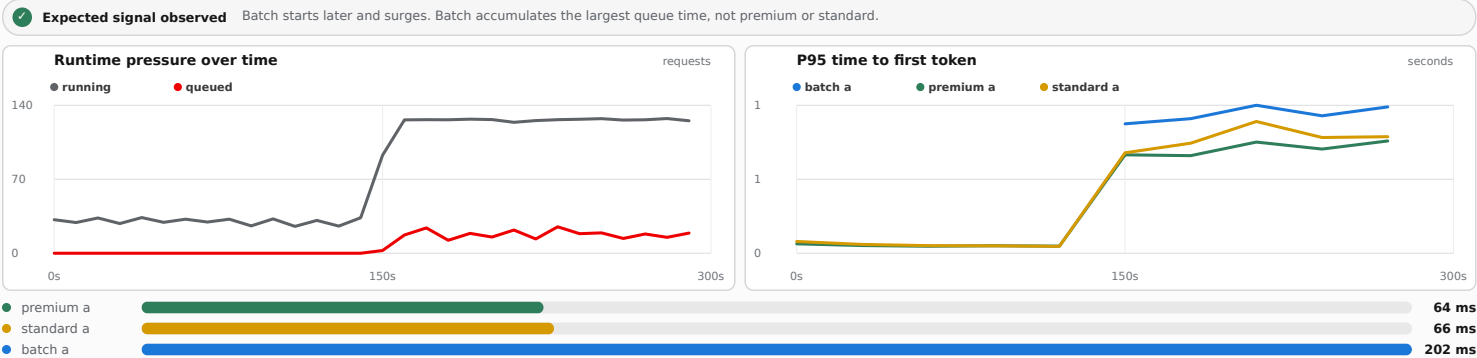


TRAFFIC	P95 TTFT	P95 LATENCY	MEAN QUEUE	MEAN 200	NON-200
● premium a intentional spike	601 ms	2,014 ms	88 ms	10,101	0
● premium b	540 ms	1,948 ms	56 ms	4,887	0
● premium c	552 ms	1,961 ms	56 ms	4,865	0

TEST 4

Priority inversion prevention

60 max queued



TRAFFIC	P95 TTFT	P95 LATENCY	MEAN QUEUE	MEAN 200	NON-200
● premium a	608 ms	2,022 ms	64 ms	3,845	0
● standard a	660 ms	2,076 ms	66 ms	3,809	0
● batch a	976 ms	2,392 ms	202 ms	10,330	2

All four scenarios matched the expected signal

The repeated runs showed the behavior each scenario was built to isolate.

- ✓ **Test 1. One endpoint, doubled load**
Premium B joined during the saturated half, and queued requests appeared.
- ✓ **Test 2. Priority differentiation**
Premium stayed steadier while standard absorbed more queue time.
- ✓ **Test 3. Same-priority fairness**
Premium A showed the spike while premium B and C stayed in the lane.
- ✓ **Test 4. Priority inversion prevention**
Batch accumulated the largest queue time during the surge.

SCENARIO	REPEATS	MEAN 200 RESPONSES	NON-200 OUTCOMES	MEAN MAX QUEUED	MEAN P95 QUEUED	MEAN MAX RUNNING	INTERPRETATION
Test 1. One endpoint, doubled load	3	22,384	0	34.7	24.7	128.0	Doubled load created queueing.
Test 2. Priority differentiation	3	17,980	0	31.3	24.0	128.0	Standard absorbed more queue time.
Test 3. Same priority fairness	3	19,853	0	32.0	25.3	128.0	Premium A spiked. B and C maintained throughput.
Test 4. Priority inversion prevention	3	17,984	2	60.3	40.3	128.0	Batch had the largest queue time.

The two non-200 outcomes were HTTP 503 responses on low-priority batch traffic during one test 4 repeat, so treat them as a caveat, not a statistically significant finding.

Run-level reconciliation

Each row shows the client request volume and maximum queued requests for that repeat.

TEST 1 One endpoint, doubled load

REPEAT	CLIENT TOTAL	MAX QUEUED
Repeat 1	22,076 client total	43.0 max queued
Repeat 2	22,514 client total	29.0 max queued
Repeat 3	22,561 client total	32.0 max queued

TEST 2 Priority differentiation

REPEAT	CLIENT TOTAL	MAX QUEUED
Repeat 1	17,969 client total	32.0 max queued
Repeat 2	17,987 client total	31.0 max queued
Repeat 3	17,983 client total	31.0 max queued

TEST 3 Same-priority fairness

REPEAT	CLIENT TOTAL	MAX QUEUED
Repeat 1	19,861 client total	32.0 max queued
Repeat 2	19,846 client total	32.0 max queued
Repeat 3	19,852 client total	32.0 max queued

TEST 4 Priority inversion prevention

REPEAT	CLIENT TOTAL	MAX QUEUED
Repeat 1	17,959 client total	64.0 max queued
Repeat 2	17,967 client total	60.0 max queued
Repeat 3	18,028 client total	57.0 max queued

How the benchmark was collected

The benchmark ran from inside the `llm-test` namespace so the traffic path matched the in-cluster gateway path.

EXECUTION	We ran a benchmark script from inside the cluster. Key flags were <code>--pressure-scenarios</code> , <code>--skip-sweep</code> , <code>--scenario-duration 300</code> , <code>--drain-timeout 120</code> , and <code>--repeats 3</code> .
WORKLOAD	The four scenario tests used the baseline profile: 512 input-token target, 128 output-token cap, and a prompt pool of 384 prompts.
ROUTING HEADERS	Each request carried the inference objective header for priority and the fairness identity header for tenant or workload identity.
MEASUREMENTS	- Client samples: status code, throughput, p95 time to first token, and p95 latency. - Endpoint-picker metrics: queue time. - vLLM metrics: running requests, queued requests, and request-success counters.
ACCOUNTING	The benchmark used a drain window after each timed scenario so in-flight requests could finish before the next repeat was summarized.
ARTIFACTS	The evidence package includes per-run summaries, tenant summaries, metric samples, and a pre-teardown Kubernetes snapshot for auditability.

Exact run configuration

KNOB	VALUE
Model	openai/gpt-oss-20b
Replicas	1
GPU	1 H100
Tensor parallelism	1
vLLM max model length	32,768 tokens
vLLM max batched tokens	8,192
vLLM max sequences	128
vLLM GPU memory utilization	0.90
Prompt pool	384 deterministic prompts
Measured input length	512 tokens
Requested output length	128 tokens
Streaming	Enabled
End-of-sequence behavior	ignore_eos=true; completions finish by length